

06 — Binary Search

DSA in Python — Interview Prep

Binary Search — Tutorial

Covers the 10 problems solved in this folder. The core idea never changes across all 10: at every step, eliminate half the remaining search space using one comparison. What changes is *what* is being searched over — an array's index, a range of possible answers, or a partition point — and that's the real skill this topic builds: recognizing when a problem has binary search's one true requirement (monotonicity - a clean "too small / too big" split) even when there's no visible sorted array to search.

The patterns

1. Exact-match search (01)

The baseline. `left <= right`, checking `nums[mid] == target` directly, closing in with `mid ± 1`. Every other pattern in this folder is a variation on this loop shape.

2. Left-boundary search (02, 03)

Instead of "does this exact value exist," the question is "what's the first index satisfying some condition." The loop shape changes subtly: `right` starts at `len(nums)` (a valid *position**, not just a valid *index**), the loop condition is `left < right`, and on a match you set `right = mid` (not `mid - 1`) because `mid` itself might be the answer and must stay eligible.

- `search_insert_position`: first index where `nums[i] >= target`.
- `first_and_last_position`: runs this shape *twice* — once biased to keep shrinking left after a match (find first), once biased to keep shrinking right (find last). Two $O(\log n)$ searches, still $O(\log n)$ total, versus an $O(n)$ scan out from one found index.

3. Search on structurally-broken sortedness (04, 05)

The array as a whole isn't sorted, but a rotation only breaks sortedness at exactly one point — so at every `mid`, at least one of the two halves is still a clean sorted run. The technique in both problems is the same: figure out *which* half is sorted by comparing boundary values, then decide which half to keep based on that.

- `search_rotated_sorted_array`: once the sorted half is known, check if target's value range falls inside it.
- `find_minimum_rotated_sorted_array`: the minimum *is* the rotation's break point, found by comparing `nums[mid]` to `nums[right]` instead of searching for a target value at all.

4. Search on slope, not on order (06)

`find_peak_element` doesn't require the array to be sorted or rotated-sorted at all — just a guarantee that a peak provably exists in whichever half is kept. Comparing `nums[mid]` to its neighbor gives a local slope; the direction of that slope tells you which half must contain a peak. This is the most abstract pattern here: binary search doesn't require *any* global order, only a local comparison that provably keeps a valid answer in the remaining half.

5. Flattened 2D search (07)

`search_2d_matrix` is pattern 1, unchanged — the only new idea is using `divmod(mid, cols)` to treat a row-major-sorted 2D grid as a single virtual 1D array, avoiding a separate row-then-column search.

6. Binary search on the answer space (08, 09)

The biggest conceptual jump in the folder: there is no array being searched at all. The *candidate answers themselves* form the search space (e.g. possible eating speeds, possible ship capacities), and what's being tested at each `mid` is a predicate — "is this candidate good enough?" — that must be monotonic (true for all values above/ below some threshold, never toggling back and forth). Any time a problem says "find the minimum/maximum X such that condition holds," and increasing X only ever makes the condition easier or only ever harder (never both), that's this pattern, regardless of whether a sorted array is anywhere in sight.

- `koko_eating_bananas`: predicate = "can she finish in $\leq h$ hours at this speed?" — monotonic because faster speed never needs more time.
- `capacity_to_ship_packages`: predicate = "can it ship in $\leq \text{days}$ days at this capacity?" — monotonic the same way.

7. Binary search on a partition point (10)

`median_of_two_sorted_arrays` is a third distinct shape: binary search over *where to cut* two arrays simultaneously so that everything left of both cuts is $\leq$ everything right of both cuts. The partition in the smaller array is searched directly; the partition in the other array is derived (`j = half - i`), not searched independently — that's what keeps the whole thing at $O(\log(\min(m,n)))$ instead of $O(\log(m) + \log(n))$ or worse. Four border values determine, in $O(1)$, whether the current partition is valid or needs to shift left/right.

Complexity summary

#	Problem	Time	Extra space
01	binary_search	$O(\log n)$	$O(1)$
02	search_insert_position	$O(\log n)$	$O(1)$
03	first_and_last_position	$O(\log n)$	$O(1)$
04	search_rotated_sorted_array	$O(\log n)$	$O(1)$
05	find_minimum_rotated_sorted_array	$O(\log n)$	$O(1)$
06	find_peak_element	$O(\log n)$	$O(1)$
07	search_2d_matrix	$O(\log(m \cdot n))$	$O(1)$
08	koko_eating_bananas	$O(n \cdot \log(\max(\text{piles})))$	$O(1)$
09	capacity_to_ship_packages	$O(n \cdot \log(\text{sum}(\text{weights})))$	$O(1)$
10	median_of_two_sorted_arrays	$O(\log(\min(m,n)))$	$O(1)$

Note 08 and 09 aren't purely $O(\log(\text{answer_range}))$ — each binary search step also runs an $O(n)$ predicate check (`hours_needed`, `days_needed`), so the total is the product, not just the log term. Interviewers will expect this distinction stated explicitly.

Common interview follow-ups to be ready for

- `binary_search`: "what if the array has duplicates?" — exact-match search still works, but "which occurrence" becomes ambiguous; that's exactly what `03` solves properly.
- `first_and_last_position`: "can you avoid writing the search logic twice?" — yes, with a single helper parameterized by a boolean (as done here) or by which comparison operator to bias toward.
- `search_rotated_sorted_array`: "what if duplicates are allowed?" — `nums[left] <= nums[mid]` can no longer reliably identify the sorted half (LC81); worst case degrades to $O(n)$.
- `find_peak_element`: "what if the array could be flat (equal neighbors) instead of strictly increasing/decreasing?" — problem guarantees strict inequality between neighbors here; discuss what breaks in the slope argument without that guarantee.
- `koko_eating_bananas` / `capacity_to_ship_packages`: "how do you even recognize this is a binary search problem?" — be ready to articulate the tell: a min/max-such-that-condition-holds question with a monotonic condition, independent of any sorted array.
- `median_of_two_sorted_arrays`: "why binary search the smaller array?" — searching the larger one is still correct but wastes time; know why the swap at the top of the function matters for the stated $O(\log(\min(m,n)))$ bound, not just for correctness.

Self-check before moving on

You should be able to, without looking at the code:

- State the difference in loop shape between exact-match search (`01`) and left-boundary search (`02`), and why each is correct for its purpose.
- Explain how a rotated array guarantees at least one sorted half at every step.
- Recognize a "binary search on the answer" problem from its description alone, without seeing the code, and state what the monotonic predicate is.
- Explain in one sentence why `median_of_two_sorted_arrays` searches a partition point instead of a value.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `07_trees`.

Programs

01. Binary Search

01_binary_search.py

Explanation

Problem: Given a sorted array of distinct integers and a target, return its index, or -1 if not present.

Source : LeetCode 704 - Binary Search

Example:

Input: nums = [-1, 0, 3, 5, 9, 12], target = 9

Output: 4

Idea: the baseline binary search. At each step, comparing the target to the middle element eliminates *half* the remaining search space, regardless of which half - that halving is what gives $O(\log n)$ instead of $O(n)$. `left <= right` (not `<`) is correct here because we're searching for an exact match and need to check the last remaining single element too.

Code

```
def binary_search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

if __name__ == "__main__":
    tests = [
        ([-1, 0, 3, 5, 9, 12], 9, 4),
        ([-1, 0, 3, 5, 9, 12], 2, -1),
        ([5], 5, 0),
        ([], 1, -1),
    ]

    for i, (nums, target, expected) in enumerate(tests, 1):
        got = binary_search(nums, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

02. Search Insert Position

Explanation

Problem: Given a sorted array of distinct integers and a target, return the index where target is found, or the index where it would be inserted to keep the array sorted.

Source : LeetCode 35 - Search Insert Position

Example:

Input: nums = [1, 3, 5, 6], target = 5

Output: 2 (found at index 2)

Input: nums = [1, 3, 5, 6], target = 2

Output: 1 (would be inserted between 1 and 3)

Idea: this is a "find the first index where `nums[i] >= target`" search, a different shape from 01's exact-match search - notice `right` starts at `len(nums)` (a valid insert position, one past the last element) rather than `len(nums) - 1`, and the loop uses `left < right` with `right = mid` (not `mid - 1`) since `mid` might itself be the answer and must stay in play. This left-boundary template is the basis for 03's first/last-occurrence search too.

Code

```
def search_insert(nums: list[int], target: int) -> int:
    left, right = 0, len(nums)

    while left < right:
        mid = (left + right) // 2
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid

    return left

if __name__ == "__main__":
    tests = [
        ([1, 3, 5, 6], 5, 2),
        ([1, 3, 5, 6], 2, 1),
        ([1, 3, 5, 6], 7, 4),
        ([1, 3, 5, 6], 0, 0),
        ([], 5, 0),
    ]

    for i, (nums, target, expected) in enumerate(tests, 1):
        got = search_insert(nums, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. First And Last Position

Explanation

Problem: Given a sorted array that may contain duplicates and a target, return [first_index, last_index] of target's occurrences, or [-1, -1] if it's not present. $O(\log n)$.

Source : LeetCode 34 - Find First and Last Position of Element in Sorted Array

Example:

Input: nums = [5, 7, 7, 8, 8, 10], target = 8

Output: [3, 4]

Idea: run a modified binary search twice. On finding a match, don't stop - record it, then keep searching in the direction that could find an *earlier* (or later) match: shrink **right** past mid to look left for the first occurrence, or push **left** past mid to look right for the last. Each of the two searches is still $O(\log n)$, so the combined cost stays $O(\log n)$, not $O(n)$ (which a linear scan out from a single found index would cost in the worst case of all-duplicates).

Code

```
def search_range(nums: list[int], target: int) -> list[int]:
    def find_bound(is_first: bool) -> int:
        left, right = 0, len(nums) - 1
        result = -1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                result = mid
                if is_first:
                    right = mid - 1
                else:
                    left = mid + 1
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return result

    return [find_bound(True), find_bound(False)]

if __name__ == "__main__":
    tests = [
        ([5, 7, 7, 8, 8, 10], 8, [3, 4]),
        ([5, 7, 7, 8, 8, 10], 6, [-1, -1]),
        ([], 0, [-1, -1]),
        ([1, 1, 1, 1], 1, [0, 3]),
    ]

    for i, (nums, target, expected) in enumerate(tests, 1):
        got = search_range(nums, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Search Rotated Sorted Array

Explanation

Problem: An array, originally sorted ascending with distinct values, has been rotated at some unknown pivot. Given the rotated array and a target, find its index in $O(\log n)$, or -1 if absent.

Source : LeetCode 33 - Search in Rotated Sorted Array

Example:

Input: nums = [4, 5, 6, 7, 0, 1, 2], target = 0

Output: 4

Idea: the array as a whole isn't sorted, but at every step, *at least one* of the two halves around `mid` is guaranteed to be a contiguous sorted run (rotation only breaks sortedness at one point). Figure out which half is sorted by comparing `nums[left]` to `nums[mid]`; if target falls within that sorted half's value range, search there; otherwise it must be in the other half. Either way, half the search space is eliminated every step, preserving $O(\log n)$.

Code

```
def search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid

        if nums[left] <= nums[mid]: # left half is sorted
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else: # right half is sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1

if __name__ == "__main__":
    tests = [
        ([4, 5, 6, 7, 0, 1, 2], 0, 4),
        ([4, 5, 6, 7, 0, 1, 2], 3, -1),
        ([1], 0, -1),
        ([3, 1], 1, 1),
        ([5, 1, 3], 5, 0),
    ]

    for i, (nums, target, expected) in enumerate(tests, 1):
        got = search(nums, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

05. Find Minimum Rotated Sorted Array

05_find_minimum_rotated_sorted_array.py

Explanation

Problem: A sorted array with distinct values has been rotated at an unknown pivot. Find the minimum element in $O(\log n)$.

Source : LeetCode 153 - Find Minimum in Rotated Sorted Array

Example:

Input: [4, 5, 6, 7, 0, 1, 2]

Output: 0

Idea: the minimum is exactly the "break point" of the rotation - the one place where a larger value is immediately followed by a smaller one. Compare `nums[mid]` to `nums[right]`: if `nums[mid] > nums[right]`, the break point (and thus the minimum) must be somewhere to the **right** of mid, since the right portion contains the wrap-around. If `nums[mid] <= nums[right]`, the right portion from mid onward is already sorted, so the minimum is at mid or to its left. Note `mid` itself is never eliminated in the second branch (`right = mid`, not `mid - 1`) since mid could BE the minimum.

Code

```
def find_min(nums: list[int]) -> int:
    left, right = 0, len(nums) - 1

    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[right]:
            left = mid + 1
        else:
            right = mid

    return nums[left]

if __name__ == "__main__":
    tests = [
        ([3, 4, 5, 1, 2], 1),
        ([4, 5, 6, 7, 0, 1, 2], 0),
        ([11, 13, 15, 17], 11),
        ([2, 1], 1),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = find_min(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

06. Find Peak Element

06_find_peak_element.py

Explanation

Problem: A peak element is one strictly greater than its neighbors (treat out-of-bounds neighbors as -infinity). Given nums (not necessarily sorted, adjacent elements never equal), find the index of *any* peak in $O(\log n)$.

Source : LeetCode 162 - Find Peak Element

Example:

Input: [1, 2, 3, 1]

Output: 2 (nums[2] = 3 is greater than both neighbors)

Idea: binary search doesn't require full sortedness, only a way to eliminate half the space each step - here that comes from slope, not order. Compare nums[mid] to nums[mid+1]: if the slope is downward (nums[mid] > nums[mid+1]), a peak is guaranteed to exist somewhere at or to the *left* of mid (values must have risen from -infinity to get there, so a local max exists on that side). If the slope is upward, a peak is guaranteed to the *right*. Either way, a peak provably exists in the kept half, so the search always terminates on a valid answer.

Code

```
def find_peak_element(nums: list[int]) -> int:
    left, right = 0, len(nums) - 1

    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[mid + 1]:
            right = mid
        else:
            left = mid + 1

    return left

def _is_peak(nums: list[int], i: int) -> bool:
    left_ok = i == 0 or nums[i - 1] < nums[i]
    right_ok = i == len(nums) - 1 or nums[i] > nums[i + 1]
    return left_ok and right_ok

if __name__ == "__main__":
    tests = [
        [1, 2, 3, 1],
        [1, 2, 1, 3, 5, 6, 4],
        [1],
        [1, 2],
        [2, 1],
    ]

    for i, nums in enumerate(tests, 1):
        idx = find_peak_element(nums)
        status = "PASS" if _is_peak(nums, idx) else "FAIL"
        print(f"Test {i}: {status} (nums={nums}, got index {idx} -> value {nums[idx]})")
```

07. Search 2D Matrix

Explanation

Problem: Given an $m \times n$ matrix where each row is sorted ascending and the first element of each row is greater than the last element of the previous row (so the whole matrix is sorted if read row by row), determine if target exists. $O(\log(m*n))$.

Source : LeetCode 74 - Search a 2D Matrix

Example:

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]],

target = 3

Output: True

Idea: the matrix's row-major layout means it's really just a sorted 1D array in disguise. Run 01's plain binary search over virtual indices $0..(\text{rows} \times \text{cols} - 1)$, converting each virtual index back to (row, col) with divmod ($\text{mid} // \text{cols}, \text{mid} \% \text{cols}$). No need to first binary-search for the row and then the column separately - one search over the flattened index space is simpler and still $O(\log(mn))$.

Code

```
def search_matrix(matrix: list[list[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False

    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1

    while left <= right:
        mid = (left + right) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target:
            return True
        elif value < target:
            left = mid + 1
        else:
            right = mid - 1

    return False

if __name__ == "__main__":
    matrix = [[1, 3, 5, 7], [10, 11, 16, 20], [23, 30, 34, 60]]
    tests = [
        (matrix, 3, True),
        (matrix, 13, False),
        ([[1]], 1, True),
        ([[1]], 2, False),
    ]

    for i, (m, target, expected) in enumerate(tests, 1):
        got = search_matrix(m, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

08. Koko Eating Bananas

08_koko_eating_bananas.py

Explanation

Problem: Koko has piles of bananas and h hours before the guards return. Each hour she picks one pile and eats up to `speed` bananas from it (if the pile has fewer than `speed`, she finishes that pile and doesn't eat more that hour). Find the minimum integer eating speed such that she finishes all piles within h hours.

Source : LeetCode 875 - Koko Eating Bananas

Example:

Input: piles = [3, 6, 7, 11], $h = 8$

Output: 4

Idea: "binary search on the answer" - a different flavor from every problem so far in this folder, all of which binary-searched over an **array index**. Here there's no array to search; instead, the answer itself (the eating speed) lives in a range $[1, \max(\text{piles})]$, and "hours needed at speed X " is **monotonic** - a faster speed never needs more hours. That monotonicity is exactly what makes binary search valid here: binary search only requires being able to evaluate "is this candidate answer good enough?" and having that answer be monotonic in the candidate - it doesn't require an actual sorted array at all.

Code

```
def min_eating_speed(piles: list[int], h: int) -> int:
    def hours_needed(speed: int) -> int:
        return sum((p + speed - 1) // speed for p in piles)

    left, right = 1, max(piles)

    while left < right:
        mid = (left + right) // 2
        if hours_needed(mid) <= h:
            right = mid
        else:
            left = mid + 1

    return left

if __name__ == "__main__":
    tests = [
        ([3, 6, 7, 11], 8, 4),
        ([30, 11, 23, 4, 20], 5, 30),
        ([30, 11, 23, 4, 20], 6, 23),
        ([312884470], 968709470, 1),
    ]

    for i, (piles, h, expected) in enumerate(tests, 1):
        got = min_eating_speed(piles, h)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

09. Capacity To Ship Packages

09_capacity_to_ship_packages.py

Explanation

Problem: Packages (given as weights, in order) must be shipped within **days** days. Each day the ship loads consecutive packages (in the given order) up to its weight capacity. Find the minimum capacity that allows shipping everything within **days** days.

Source : LeetCode 1011 - Capacity To Ship Packages Within D Days

Example:

Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5

Output: 15

Idea: same "binary search on the answer" shape as 08. The candidate answer is the ship's capacity, ranging from $\max(\text{weights})$ (must fit the single heaviest package) to $\sum(\text{weights})$ (ship everything in one day). "Days needed at capacity X" is monotonic - a bigger capacity never needs more days - so binary search for the smallest capacity whose days-needed is still $\leq$ the limit.

Code

```
def ship_within_days(weights: list[int], days: int) -> int:
    def days_needed(capacity: int) -> int:
        trips = 1
        current_load = 0
        for w in weights:
            if current_load + w > capacity:
                trips += 1
                current_load = 0
            current_load += w
        return trips

    left, right = max(weights), sum(weights)

    while left < right:
        mid = (left + right) // 2
        if days_needed(mid) <= days:
            right = mid
        else:
            left = mid + 1

    return left

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5, 15),
        ([3, 2, 2, 4, 1, 4], 3, 6),
        ([1, 2, 3, 1, 1], 4, 3),
    ]

    for i, (weights, days, expected) in enumerate(tests, 1):
        got = ship_within_days(weights, days)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

10. Median Of Two Sorted Arrays

10_median_of_two_sorted_arrays.py

Explanation

Problem: Given two sorted arrays nums1, nums2, return the median of the combined sorted array, in $O(\log(\min(m, n)))$ time.

Source : LeetCode 4 - Median of Two Sorted Arrays

Example:

Input: nums1 = [1, 3], nums2 = [2]

Output: 2.0

Input: nums1 = [1, 2], nums2 = [3, 4]

Output: 2.5

Idea: the hardest problem in this folder. Merging the two arrays and taking the middle would be $O(m+n)$ - too slow for the required $O(\log(\min(m,n)))$. Instead, binary search for a *partition point* i in the smaller array (always search the smaller one, for efficiency): given i , the corresponding partition j in the other array is forced ($j = \text{half} - i$, where half is how many elements the "left side" of the combined median split needs). A valid partition requires every element left of the cut to be $\leq$ every element right of the cut, across *both* arrays simultaneously - checked with 4 border values (left_a, right_a, left_b, right_b). If left_a > right_b, the partition in nums1 needs to move left; if left_b > right_a, it needs to move right. This is binary searching over partition positions, not over values - a third distinct "shape" of binary search alongside the array-index searches (01-07) and the answer-space searches (08-09).

Code

```
def find_median_sorted_arrays(nums1: list[int], nums2: list[int]) -> float:
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    left, right = 0, m
    half = (m + n + 1) // 2

    while left <= right:
        i = (left + right) // 2
        j = half - i

        left_a = nums1[i - 1] if i > 0 else float("-inf")
        right_a = nums1[i] if i < m else float("inf")
        left_b = nums2[j - 1] if j > 0 else float("-inf")
        right_b = nums2[j] if j < n else float("inf")

        if left_a <= right_b and left_b <= right_a:
            if (m + n) % 2 == 1:
                return float(max(left_a, left_b))
            return (max(left_a, left_b) + min(right_a, right_b)) / 2
        elif left_a > right_b:
```

```

        right = i - 1
    else:
        left = i + 1

    raise ValueError("Input arrays must be sorted")

if __name__ == "__main__":
    tests = [
        ([1, 3], [2], 2.0),
        ([1, 2], [3, 4], 2.5),
        ([], [1], 1.0),
        ([2], [], 2.0),
        ([0, 0], [0, 0], 0.0),
    ]

    for i, (a, b, expected) in enumerate(tests, 1):
        got = find_median_sorted_arrays(a, b)
        status = "PASS" if abs(got - expected) < 1e-9 else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```